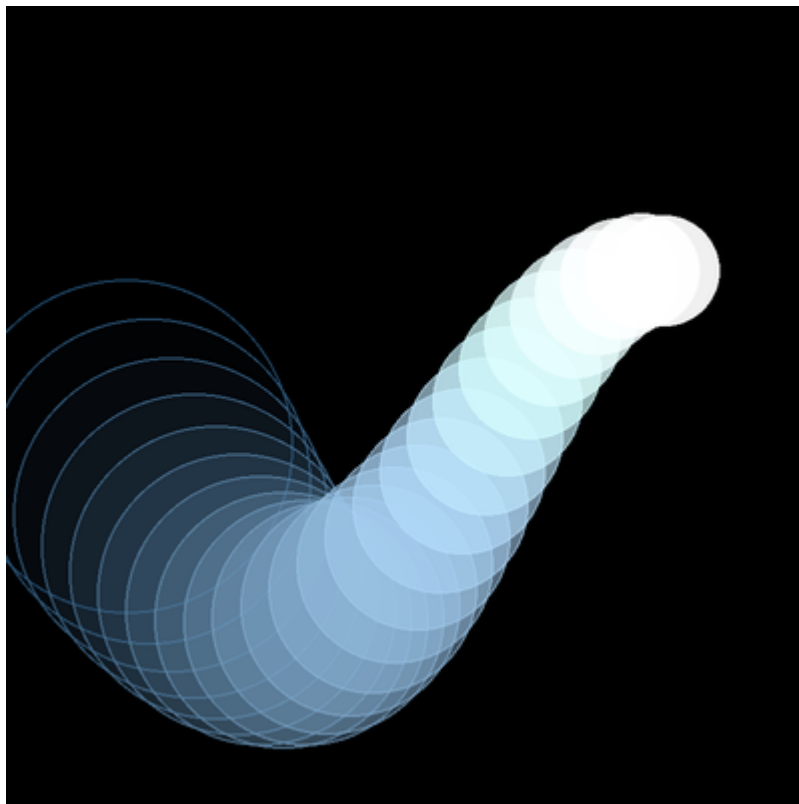


Cours 08 — Traînée évoluée : couleur, transparence, taille

Avant : cours 07 (fonctions, file de positions), cours 06 (temps). **Comment travailler** : toujours dans `ofApp.h` et `ofApp.cpp`, par versions successives. `ofApp08.h` / `ofApp08.cpp` : la référence téléchargeable de l'état final.

Ce cours est le **corrigé de l'Autonomie du cours 07**. C'est une solution possible, pas la seule : si la tienne fait autre chose et que les contraintes sont respectées, elle est aussi valable.

La traînée du cours 07, mais chaque élément a sa propre taille, sa propre transparence et sa propre couleur, calculées à partir de son **indice** dans la file et du **temps**. Rien de nouveau dans les outils : c'est un cours sur ce qu'on peut faire en **combinant** ceux qu'on a.



■ 1. Étape 1 — le temps : une couleur qui vit

Repars de la traînée du cours 07, en remplaçant l'ours par un simple cercle, avec une file de 25 (`prevX`, `prevY` dans le `.h`, plus `float t { 0 };` et `float r { 0 }, g { 0 }, b { 0 };`). Dans `update()` :

```
t = ofGetElapsedTimef();
r = (cos(t * 1.2f) / 2 + 0.5f) * 255;
g = (cos(t * 1.0f) / 2 + 0.5f) * 255;
b = (cos(t * 0.86f) / 2 + 0.5f) * 255;
```

Et dans `draw()`, `ofSetColor(r, g, b)` avant les cercles : toute la traînée change de couleur en continu.

`ofGetElapsedTimef()` : le temps écoulé depuis le lancement, en secondes. C'est l'**autre** façon d'animer, à côté de la vitesse du cours 06 : au lieu d'ajouter un pas à chaque frame, on **calcule directement** une valeur à partir du temps.

La formule, enfin expliquée : `cos(t)` oscille entre -1 et 1 quand `t` avance ; `/ 2 + 0.5f` le ramène entre 0 et 1 ; `* 255` entre 0 et 255 . Le nombre qui multiplie `t` est la **vitesse d'oscillation** — trois vitesses différentes pour `r`, `g`, `b`, et la couleur se promène sans jamais se répéter.

Essaie : change les trois vitesses (`1.2f`, `1.0f`, `0.86f`). Avec trois vitesses **égales**, que devient le fond de couleurs ? (Réponse attendue : un simple gris qui pulse — pourquoi ?)

■ 2. Étape 2 — un nombre qui dépend de l'indice

La variable-clé du cours, dans la boucle de dessin :

```
for (int i = 0; i < prevX.size(); i++) {  
    float offset = i * 10;  
  
    ofFill();  
    ofSetColor(r + offset, g + offset, b + offset, offset);  
    ofDrawCircle(prevX[i], prevY[i], 40);  
  
    ofNoFill();  
    ofSetColor(r + offset, g + offset, b + offset, 255 - offset);  
    ofDrawCircle(prevX[i], prevY[i], 40);  
}
```

`i` va de 0 à 24 , donc `offset` va de 0 à 240 . Tout ce qui doit changer **le long** de la traînée se calcule à partir de lui :

- le remplissage a pour transparence `offset` : le plus ancien (`i = 0`) est invisible, le plus récent presque opaque ;
- le contour a pour transparence `255 - offset` : l'**inverse** — les anciens n'ont qu'un contour, les récents un plein ;
- la couleur est éclaircie de `offset` : les récents tirent vers le blanc.

Deux passes fill / contour (cours 02) avec des transparences **croisées** : c'est ça qui donne la profondeur.

Une remarque pour plus tard : `r + offset` peut dépasser 255 . Ici le sketch compte sur openFrameworks pour arrondir, mais ce n'est pas garanti — au cours 10 on apprendra à **borner** proprement avec `ofClamp`.

Essaie : `offset / 2` à la place d' `offset` dans la transparence. Puis remplace `(255 - offset)` par `offset` : que devient l'effet de profondeur ?

■ 3. Étape 3 — ranger la recette dans une fonction

La boucle devient illisible ? Range l'aspect d'un élément dans une fonction — c'est le geste du cours 07, avec plus de paramètres :

```
// i : indice dans la traînée (0 = le plus ancien). t : temps en secondes.
void ofApp::forme(float x, float y, int i, float t, float r, float g, float b) {
    float offset = i * 10;

    ofFill();
    ofSetColor(r + offset, g + offset, b + offset, offset);
    ofDrawCircle(x, y, 40);

    ofNoFill();
    ofSetColor(r + offset, g + offset, b + offset, 255 - offset);
    ofDrawCircle(x, y, 40);
}

void ofApp::draw() {
    ofBackground(0);
    for (int i = 0; i < prevX.size(); i++) {
        forme(prevX[i], prevY[i], i, t, r, g, b);
    }
}
```

(Et l'annonce dans le `.h`, comme toujours.) Sept paramètres : la fonction reçoit **tout ce dont elle a besoin** — position, indice, temps, couleur de base — et n'invente rien. Le point important : elle ne décide pas de son aspect toute seule, elle le **déduit** de `i` et de `t`. Le même code dessine 25 cercles tous différents.

Essaie : remplace le cercle par l'ours du cours 07, en lui ajoutant un paramètre de taille pour que tout soit proportionnel.

■ 4. Étape 4 — combiner indice et temps

La taille, dernière pièce — dans `forme()` :

```
float taille = (cos(i / 5.0f) + 5) * 20;
float variation = (cos(t * 1.2f) + 5) * 100;
float tailleFinale = taille + variation * (25 - i) / 50.0f;
ofDrawCircle(x, y, tailleFinale / 2);
```

- `taille` dépend de `i` : elle ondule **le long** de la traînée ;
- `variation` dépend de `t` : elle pulse **dans le temps** ;
- $(25 - i) / 50.0f$ vaut 0.5 pour le plus ancien et presque 0 pour le plus récent : la pulsation est plus forte à l'arrière.

Trois lignes, et chaque cercle a une taille qui dépend à la fois de sa place et du moment. Il n'y a pas de recette magique : on assemble des morceaux qui varient entre des **bornes connues**, et on regarde. (Les `5.0f` et `50.0f` évitent la division entière — cours 01.)

Essaie : invente **ta** formule de taille. Avant de la taper, note sa valeur pour `i = 0` et `i = 24` — la discipline de l'Autonomie du cours 07.

■ 5. Ce que ce cours prépare

On a fabriqué une couleur avec trois cosinus. Ça marche, mais on ne contrôle rien : on ne sait pas quelle teinte vient ensuite, et la couleur passe par des gris. Le cours 09 donne un outil bien plus direct : décrire une couleur par sa **teinte**.

■ Exercices

1. **Lire avant de lancer** — si on remplace `float offset = i * 10;` par `float offset = (25 - i) * 10;`, dans quel sens la traînée s'estompe-t-elle ? Réponds, puis vérifie.
2. **Pair, impair** — un cercle pour `i` pair, un carré pour `i` impair : `if (i % 2 == 0)`. Le `%` donne le reste de la division — il vaut 0 une fois sur deux.
3. **Le feu follet** — la pulsation temporelle appliquée à la **transparence du contour** au lieu de la taille : une traînée qui bat comme un cœur, sans changer de forme.
4. **La traînée miroir** (*plus costaud*) — dessine la file une deuxième fois, symétrique (`800 - x`), avec une formule d'aspect **différente** (autre taille, autres transparences). Deux créatures liées par la même mémoire.

■ Ce qu'il faut retenir

- `ofGetElapsedTimef()` : le temps en secondes — animer en **calculant depuis le temps**, l'alternative à la vitesse du 06.

- $\cos(t * \text{vitesse}) / 2 + 0.5f$: une oscillation douce entre 0 et 1, à la vitesse choisie.
- Tout ce qui varie le long d'une file se calcule depuis l'**indice** `i` ; tout ce qui varie au fil du temps, depuis `t` — et on peut combiner les deux.
- Une fonction de dessin reçoit **tout** en paramètres et déduit son aspect : le même code, vingt-cinq résultats différents.
- Assembler des formules = connaître leurs **bornes** avant de les taper.